

Advanced Algorithms and Data Structures

Lecture# 05: Hashing

Searching

- Organizing and retrieving information is at the heart of most computer applications
- We can define searching formally as follows
 - Suppose that we have a collection L of n records of the form
$$(k_1, I_1), (k_2, I_2), \dots, (k_n, I_n)$$
 - Given a particular key value K , the search problem is to locate record (k_j, I_j) in L such that $k_j = K$ (if one exists)
- An **exact-match query** is a search for the record whose key value matches a specified key value
- A **range query** is a search for all records whose key value falls within a specified range of key values

Sequential Search (Linear Search)

- Sequential search on an unsorted list requires $O(n)$ time in the worst case
- How many comparisons does linear search do on average searching whether K is in list L ?
- We have

$$P(K \notin L) = 1 - \sum_0^{n-1} P(K = L[i])$$

Where $P(x)$ is the probability of event x

- Let p_i be the probability that K is in position i of L and p_n be the probability that K is not in L . The average cost $T(n)$:

$$T(n) = np_n + \sum_0^{n-1} (i + 1)p_i$$

Sequential Search (cont.)

- If we assume that all p_i are equal ($p_i = p$) except p_n

$$\begin{aligned} T(n) &= np_n + \sum_{i=0}^{n-1} (i+1)p \\ &= \frac{n+1+p_n(n-1)}{2} \end{aligned}$$

- Depending on the value of p_n

$$\frac{n+1}{2} \leq T(n) \leq n$$

Jump Search

- Given a sorted array L , for some value of j , we check every j^{th} element in L
- So long as K (the value we want to find) is greater than the values we are checking, we continue on
- When we reach value in L greater than K , we do a linear search on the piece of length $j - 1$
- If we define m such that $mj \leq n < (m + 1)j$, the cost to run the algorithm on n items with a jump size j is

$$T(n, j) = m + j - 1 = \left\lfloor \frac{n}{j} \right\rfloor + j - 1 \text{ (3-way comparisons)}$$

Jump Search (cont.)

- What is the best value that we can pick for j ?
 - Take the derivative and solve for $T'(j) = 0$ to find the minimum, which is $j = \sqrt{n}$
- In this case, the worst case cost will be roughly $2\sqrt{n}$

Binary search

- Given a sorted array, binary search:
 - looks at the middle element and determines if the value being search for is in the left half or the right half of the array
 - Continues splitting the appropriate subarray in half until the

desiPosition	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Key	11	13	21	26	29	36	40	41	45	51	54	56	65	72	77	83



- How many times can an array of size n be split in half until only one element remains in the final sub array?

Interpolation Search

- If we know nothing about the distribution of key values, then binary search is the best algorithm available for searching a sorted array
 - But sometimes we do know something about the expected key distribution $\Rightarrow$ dictionary search
- In an interpolation (dictionary) search, to split the list:

$$mid = Lo + \frac{(Hi - Lo) * (X - A[Lo])}{A[Hi] - A[Lo]}$$

where –

`A = list`

`Lo = Lowest index of the list`

`Hi = Highest index of the list`

`A[n] = Value stored at index n in the list`

Interpolation Search

- Searching Algorithm:
 1. Start searching data from middle of the list.
 2. If it is a match, return the index of the item, and exit.
 3. If it is not a match, probe position.
 4. Divide the list using probing formula and find the new middle.
 5. If data is greater than middle, search in higher sub-list.
 6. If data is smaller than middle, search in lower sub-list.
 7. Repeat until match.

Time Complexity

Searching Algorithm	Best case	Average Case	Worst Case
Linear Search	$O(1)$	$O(n)$	$O(n)$
Jump Search	$O(1)$	$O(\sqrt{n})$	$O(\sqrt{n})$
Binary Search	$O(1)$	$O(\log_2 n)$	$O(\log_2 n)$
Interpolation Search	$O(1)$	$O(\log(\log(n)))$	$O(n)$

Hashing

- Different approach to searching arrays by ***direct access based on key value***
- The process of finding a record using some computation to map its key value to a position in the array is called **hashing**
- Most hashing schemes place records in the array in whatever order satisfies the needs of the address calculation, thus the records are **not** ordered by:
 - Value
 - Frequency

Terminologies

- The function that maps key values to positions is called a **hash function** and will be denoted by h
- The array that holds the records is called the **hash table** and will be denoted by HT
- A position in the hash table is also known as a **slot**
- The number of slots in hash table HT will be denoted by the variable M , with slots numbered from 0 to $M - 1$
- The goal for a hashing system is to arrange things such that, for any key value K and some hash function h , $i = h(K)$ is a slot in the table such that $0 < h(K) < M$, and we have the key of the record stored at $HT[i]$ equal to K

Simple Example

- Consider storing n records each with a unique key value in the range 0 to $n - 1$
- A record with key K can be stored in $HT[k]$
- The hash function is simply $h(k) = k$
- To find the record with key value K , simply look in
 $HT[h(k)] = HT[k]$

Slot Collision

- Typically, there are many more values in the key range than there are slots in the hash table

⇒ At least some of the slots must be mapped to form multiple key values

- Given a hash function h and two keys k_1 and k_2 , if $h(k_1) = \beta = h(k_2)$ where β is a slot in the table, then we say that k_1 and k_2 have a **collision** at slot under hash function h

Record Finding Procedure

- Finding a record with key value K in a database organized by hashing follows a two-step procedure:
 1. Compute the table location $h(K)$.
 2. Starting with slot $h(K)$, locate the record containing key K using (if necessary) a **collision resolution policy**.

Hash Functions

- We would like to pick a hash function that stores the actual records in the collection such that each slot in the hash table has equal probability of being filled
- In many application the incoming records are poorly distributed
- When designing hash functions, we are generally faced with one of two situations:
 - We know nothing about the distribution of the incoming keys: the hash function should evenly distribute the key range across the hash table
 - We know something about the distribution of the incoming keys: the hash function should be distribution-dependent

Example 1

- Consider the following hash function used to hash integers to a table of sixteen slots:

```
int h(int x) {  
    return (x % 16);  
}
```

- The result may also be poorly distributed since it depends only the last 4 bits of the key

Example 1 (Better Hash Function)

- A good hash function for numerical values comes from the **mid-square** method
- The mid-square method squares the key value, and then takes the middle r bits of the result, giving a value in the range 0 to $2^r - 1$
- Example:
 - Consider records whose keys are 4-digit numbers in base 10
 - The goal is to hash these key values to a table of size 100
 - This range is equivalent to two digits in base 10, that is $r = 2$
 - If the input is the number 4567, squaring yields an 8-digit number, 20857489. The middle two digits of this result are 57

```
  4567
  4567
-----
 31969
 27402
 22835
 18268
20857489
-----
  57
```

Example 2

- Here is a hash function for strings of characters:

```
int h(char *ch, int M) {  
    int i, sum;  
    for (sum=0, i=0; i<x.length(); i++)  
        sum += ch[i];  
    return sum % M;  
}
```

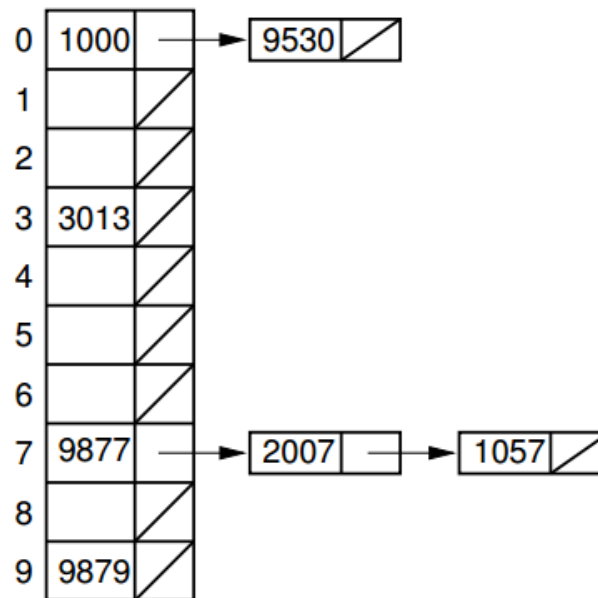
- This is an example of the **folding** approach to designing a hash function
 - breaks up a key value into precise segments that are added to form a hash value

Collision Resolution Policy

- Collision resolution techniques can be broken into two classes:
 - open hashing (also called separate chaining)
 - closed hashing (also called open addressing)
- The difference between the two has to do with whether collisions are stored outside the table (open hashing), or whether collisions result in storing one of the records at another slot in the table (closed hashing)

Open Hashing

- The simplest form of open hashing defines each slot in the hash table to be the head of a linked list



Closed Hashing

- Closed hashing stores all records directly in the hash table
- Each record R with key value k_R has a home position that is $h(k_R)$, the slot computed by the hash function.
- If R is to be inserted and another record already occupies R 's home position, then R will be stored at some other slot in the table
- It is the business of the collision resolution policy to determine which slot that will be

Bucket Hashing

- One implementation for closed hashing groups hash table slots into **buckets**
- The M slots of the hash table are divided into B buckets, with each bucket consisting of M/B slots
- The hash function assigns each record to the first slot within one of the buckets
- If this slot is already occupied, then the bucket slots are searched sequentially until an open slot is found
- If a bucket is entirely full, then the record is stored in an **overflow bucket** of infinite capacity at the end of the table

Hash Table		Overflow
0	1000	1057
	9530	
1		
2	9877	
	2007	
3	3013	
4	9879	

Linear Probing

- Closed hashing with no bucketing, and a collision resolution policy that can potentially use any slot in the hash table
- We can view any collision resolution method as generating a sequence of hash table slots
- This sequence of slots is known as the **probe sequence** and it is generated by some **probe function** that we will call p

Linear Probing (cont.)

- The probe function for simple linear probing is

$$p(K, i) = i$$

- That is, the i^{th} offset on the probe sequence is just i , meaning that the i^{th} step is simply to move down i slots in the table
- Once the bottom of the table is reached, the probe sequence wraps around to the beginning of the table

Linear Probing (cont.)

- **Problem:** primary clustering

0	9050
1	1001
2	
3	
4	
5	
6	
7	9877
8	2037
9	

(a)

0	9050
1	1001
2	
3	
4	
5	
6	
7	9877
8	2037
9	1059

(b)

Improved Collision Resolution Methods

How can we avoid primary clustering?

- One possible improvement might be to use linear probing, but to skip slots by a constant c other than 1
 - This would make the probe function $p(K, i) = ci$
 - The i^{th} slot in the probe sequence will be $(h(K) + ci) \bmod M$
- Pseudo-random probing
 - the i^{th} slot in the probe sequence is $(h(K) + r_i) \bmod M$ where r_i is the i^{th} value in a random permutation of the numbers from 1 to $M - 1$
 - The probe function $p(K, i) = Perm[i - 1]$ where Perm is an array of length $M - 1$ containing a random permutation of the values from 1 to $M - 1$
- Other methods: quadratic probing, double hashing, ...etc

Quadratic Probing

- Quadratic probing is a method with the help of which we can solve the problem of clustering that was discussed above.
- This method is also known as the **mid-square** method. In this method, we look for the **i^2 th** slot in the **i^{th}** iteration.
- We always start from the original hash location. If only the location is occupied then we check the other slots.

$$h_i(K) = (H(K) + i^2) \% S$$

where $h_i(K)$ is the next index for the key K .

i is the collision number.

Double Hashing

- Double hashing is a technique that reduces clustering in an optimized way.
- In this technique, the increments for the probing sequence are computed by using another hash function.
- We use another hash function $\text{hash2}(x)$ and look for the $i \cdot \text{hash2}(x)$ slot in the i^{th} rotation.

$$h(x) = \text{hash}(x) + i \cdot \text{hash2}(x) \% S$$

Deletion

- When deleting records from a hash table, there are two important considerations:
 - Deleting a record must not hinder later searches
 - We do not want to make positions in the hash table unusable because of deletion
- Both of these problems can be resolved by placing a special mark in place of the deleted record, called a **tombstone**
 - If a tombstone is encountered when searching along a probe sequence, the search procedure continues with the search
 - When a tombstone is encountered during insertion, that slot can be used to store the new record

References

- Clifford A. Shaffer. (2012). *Data Structures and Algorithm Analysis*. Department of Computer Science. Virginia Tech.
- Douglas Wilhelm Harder. (2013). *Queues*. University of Waterloo, Waterloo, Ontario.
- Kurt Mehlhorn and Peter Sanders. (2007). *Algorithms and Data Structures*. Springer.
- Naveen Garg. (2008). *Data Structures and Algorithms*. Department of Computer Science and Engineering. Indian Institute of Technology, Delhi.
- Robert Lafore. (2003). *Data Structures & Algorithms in Java*. Sams Publishing.